

Quantum Circuits Learner's Space

01 - Course Intro

"The best way to learn to ride a bicycle is to actually get on the saddle and try your hands at riding it"

This Quantum Circuit's Learner's Space aims to teach you how Quantum Computers work by getting you hands-on with the underlying math and code.

In this 4-week course, you'll learn how computation happens, build a decomposition algorithm, and finally test it on Grover's Algorithm. As a result, you'll learn how scientific libraries like Qiskit work under the hood.

But first, an intro to Quantum Computing.

- [01 - Course Intro](#)
 - [Why Quantum? - Cutting through the Hype](#)
 - [Classical Computation Recap](#)
 - [Quantum Circuits - how a computation is represented](#)
 - [Quantum Technologies - how qubits work physically](#)
 - [Matrix Decomposition](#)

Why Quantum? - Cutting through the Hype

1. A Quantum Computer is a device which exploits quantum phenomena like superposition and entanglement to perform computations faster. Quantum Computers are not faster than classical computers at all tasks, rather, they are specially designed to perform certain tasks exponentially faster. Hence, your laptop in 10 years will not be quantum, but quantum computers will solve certain hard tasks important for humanity.
2. When you send message to your friend on WhatsApp, the messages are encrypted - meaning a third party intercepting the communication can't read your message.
3. This encryption relies on the key fact that **factoring large numbers is hard for a classical computer**. By "hard" we mean the time it takes to factor a number of N bits is exponential in N : denoted as $O(e^N)$. Because the N used in encryption is in the thousands, it would take a classical computer thousands of years to break it, which we simplify by calling impossible.
4. The architecture of Quantum Computers (QC) means they can factor in $O(N^3)$ time: polynomial instead of exponential! Theoretically, a Quantum Computer could break encryption and read encrypted communications in a useful duration of time.
5. This is one of the reasons governments, industry, and academia are investing billions into developing Quantum Computers. There is no doubt that Quantum Computing will be a transformational technology in our lifetime. Quantum Computers could also tackle similar computationally hard problems of societal relevance: **drug discovery** by simulating complex biomolecules, **finding better materials** like electrolytes and

superconductors, and running faster **machine learning models** on the path to Artificial General Intelligence (AGI).

6. However, we must be wary of this hype train: while a Quantum Computer *could* break encryption, current implementations suffer from fundamental architectural problems. To solve the tasks like encryption, Quantum Computers would need millions of qubits and gates. **This Learner's Space will teach you why so many gates are needed, and why Quantum Computers are so limited.**

Classical Computation Recap

Everyday ("classical") computers are digital logic circuits:

1. the data can take only a **discrete** and finite set of values
2. binary logic is dominant because of its simplicity: the smallest unit of data (bit) can equal 0 or 1
3. this data is stored in **memory**, for example the 16GB of RAM in your laptop
4. this data is transformed by **gates**, which change the value of the bits according to a definite rule (the truth table of the gate): for example AND, OR, NOT, NAND
5. **computation** means performing a well-defined calculation by repeatedly acting gates on bits. For example, an **adder** is a **circuit of gates** which takes in two numbers and outputs their sum.
6. However this discrete nature makes it hard for classical computers to solve continuous problems: for example when simulating a molecule, the molecule has hundreds of continuous degrees-of-freedom: each atom can have a continuous position and rotation.

Quantum Circuits - how a computation is represented

1. Quantum Computers are inherently continuous: the smallest unit of data is a **qubit**.
2. We can represent a quantum memory, i.e. a set of n qubits, as a vector of 2^n complex numbers
3. Quantum logic gates which act on the qubits are simply matrices of size $2^n \times 2^n$ acting on the vector.
4. Just like in classical computers, we arrange the gates in a circuit (a **Quantum Circuit**) to perform a desired computation, for example addition.
5. A typical quantum computation consists of preparing qubits in some state, for example all zeroes, then passing them through the quantum circuit, and measuring the results at the end.
6. Because of Quantum Mechanics, while a qubit in the circuit has a definite, continuous value, measuring a qubit results in one of many discrete values, with certain probabilities. The complex values of the qubits control the probability distribution (we will understand this more later).
7. In this course, we will only worry about the circuit itself, in which the qubit has a definite, continuous value represented by a vector of complex numbers.

Quantum Technologies - how qubits work physically

1. Classical computers can be implemented using different technologies: for example a 0-or-1 bit can be represented by the charge of a capacitor being low or high (like the RAM in your laptops) or by a magnet whose dipole points up or down (like Hard Disks).
2. Similarly, actual quantum computers use different technologies for memory and compute. For example:
3. Superconducting qubits have the current in a ring being clockwise or anticlockwise. Gates use microwave pulses to change the state of the qubits.
4. Semiconductor quantum dots have an electron's spin being up or down. Oscillating magnetic fields act as gates.
5. **These gate technologies all have restrictions** - they can only implement certain operations. This is just like how classical chips may only have NAND gates, and the circuit designer has to convert their circuit (composed of AND, OR, and NOT gates) into purely NAND gates: this is called **decomposition**.
6. **The restriction of Quantum Computers to a small set of gates is the reason that even simple operations take millions of gates, and hence practical Quantum Computing is difficult.**
7. Specifically, gates are lossy - they have some error, and that error accumulates over the millions of gates leading to useless noise.

Matrix Decomposition

1. When we have a desired computation like addition, we have a matrix of size $2^n \times 2^n$.
2. However, actual gate technologies may only support some basic gates acting on two qubits.
3. **Your task in this course is to implement an algorithm to break an arbitrary matrix into a circuit of basic gates.**
4. This will teach you both how quantum circuits work, and why Quantum Computing is such a hard problem.
5. A lot of the research conducted by Quantum Computing companies like Google and IBM is about finding more efficient decomposition algorithms which use fewer gates.
6. Popular Quantum Computing libraries like Qiskit run these decomposition algorithms everytime you run their code.
7. Hence it is extremely valuable to understand how these decomposition algorithms work.
8. This course explicitly doesn't use Qiskit - that would be like running `from torch import nn` i.e. using PyTorch to understand how a Neural Network works.

Happy Computing!